

GNU/Linux

Лаба №6: Индивидуальные задания

Железняков Марк

@mrqiz

Формат *DWARF* используется в ядре Linux для **отладки на уровне исходного кода**.

Исключение отладочных данных из ядра может помочь сделать процесс сборки быстрее, а ядро - легче.

```
scripts/config -d CONFIG_DEBUG_INFO_DWARF_TOOLCHAIN_DEFAULT
```

исключить DWARF из ядра

```
cat /boot/config-X.Y.Z | grep CONFIG_DEBUG_INFO_NONE
```

проверка отключения символьной информации

Сборка без символьной информации явно подразумевается в вариантах 1, 2, 3, 4 и 7.

Перед выполнением работы необходимо отключить опции отладки ядра (см. слайд 2)

Задача: сравнить характеристики ядра (время сборки и размер) с символьной информацией и без неё. Найти разницу между ними.

```
du /boot/vmlinuz-X.Y.Z
```

получение размера ядра

du выводит количество **блоков**, используемые файлами/директориями.
Обычно, 1 блок = 1024 байта

| *Перед выполнением работы необходимо отключить опции отладки ядра (см. слайд 2)*

| *Сборка с помощью кэширования может помочь ускорить повторную компиляцию.*

Задача: Скомпилировать ядро без дополнительных опций. Затем, провести снова повторную компиляцию без аргументов и не очищая (make clean) дерево сборки. В конце, собрать ядро с помощью ccache. Сравнить время трех сборок.

```
time KBUILD_BUILD_TIMESTAMP='' make CC="ccache gcc" -j8
```

 измерение времени сборки с помощью ccache

Перед выполнением работы необходимо отключить опции отладки ядра (см. слайд 2)

Монтирование каталогов как TMPFS - это эффективный способ ускорения доступа к своим файлам.

Задача: Собрать ядро в дереве tmpfs. Сравнить время сборки в нем с обычным временем сборки.

```
sudo mount -t tmpfs -o size=30G tmpfs DIRNAME_HERE
```

монтирование директории для работы с tmpfs

| *Перед выполнением работы необходимо отключить опции отладки ядра (см. слайд 2)*

Задача: Собрать ядро с использованием каналов вместо временных файлов.

```
KCFLAGS="-pipe" make -j8
```

сборка

Задача:

- Собрать ядро, используя исходный код, предоставляемый дистрибутивом;
- Используя исходный код, предоставляемый ресурсом kernel.org, с наложением на него заплат от дистрибутива.

```
apt source linux
```

получение исходного кода ядра linux

наложение патчей ядра Debian на ядро с kernel.org

```
for P in $(ls PATH/T0/DEB/KERNEL/debian/patches/**/*.patch); do patch -p1 < $P; done
```

Задача: Сменить активное ядро без перезагрузки системы.

```
sudo apt install -y kexec-tools
```

```
sudo kexec -l /boot/vmlinuz-X.Y.Z --initrd=/boot/initrd.img-X.Y.Z --reuse-commandline
```

 загрузка ядра

```
sudo kexec -e
```

 запуск загруженного ядра

Перед выполнением работы необходимо отключить опции отладки ядра (см. слайд 2)

Задача: Собрать ядро без использования символьной информации под вашу целевую платформу.

```
sudo apt install -y gcc-aarch64-linux-gnu
```

установка gcc под целевую платформу

```
make ARCH=arm CROSS_COMPILE=aarch64-linux-gnu- menuconfig
```

menuconfig с опциями целевой платформы

```
make ARCH=arm CROSS_COMPILE=aarch64-linux-gnu- -j8
```

сборка ядра

Задача: Используя git, скачать исходный код ядра, предоставляемый ресурсом kernel.org, наложить на него заплаты с использованием git, собрать ядро с патчами.

```
sudo apt install -y git
git clone https://github.com/torvalds/linux.git
git checkout BRANCHNAME
```

клонирование репозитория ядра
переключение активной ветки

наложение патчей ядра Debian на ядро с kernel.org средствами Git

```
for P in $(ls PATH/T0/DEB/KERNEL/debian/patches/**/*.*.patch); do git apply $P; done
```